

Αναφορά πάνω στην 3η εργασία

#####

AEM: ####

30/11/2024

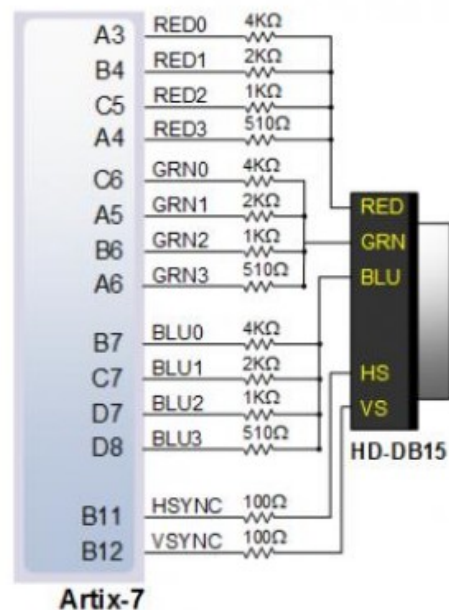
• Περίληψη:

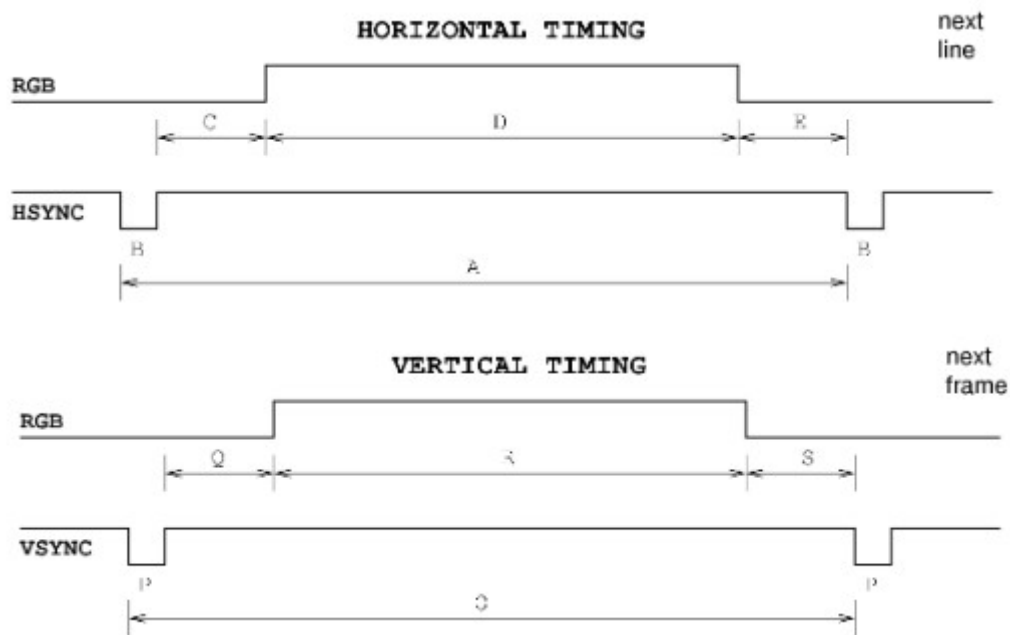
Στην αναφορά αυτή παρουσιάζεται η υλοποίηση ενός ελεγκτή οθόνης που χρησιμοποιεί το VGA standard και σύνδεση. η εξήγηση και λεπτομέρειες αυτής, καθώς και λεπτομέρειες σχετικά με την επαλήθευση σωστής λειτουργίας τους. Συγκεκριμένα παρουσιάζεται η υλοποίηση VRAM για την αποθήκευση μίας εικόνας, μονάδων συγχρονισμού των σημάτων συγχρονισμού VSYNC και HSYNC που χρησιμοποιεί το VGA, και περιφεριακά κομμάτια της που επιτρέπει τη σωστή λειτουργία τους.

• Εισαγωγή:

Όπως αναφέρθηκε προηγουμένως, κύριος στόχος της εργασίας είναι η υλοποίηση ενός ελεγκτή VGA, το οποίο περιλαμβάνει την υλοποίηση μιας μνήμης VRAM που θα περιέχει μία εικόνα, μονάδων που οδηγούν σήματα συγχρονισμού HSYNC και VSYNC, καθώς και καθορίζουν την διεύθυνση από την οποία θα λάβουμε δεδομένα, με τη χρήση μετρητών HPIXEL και VPIXEL.

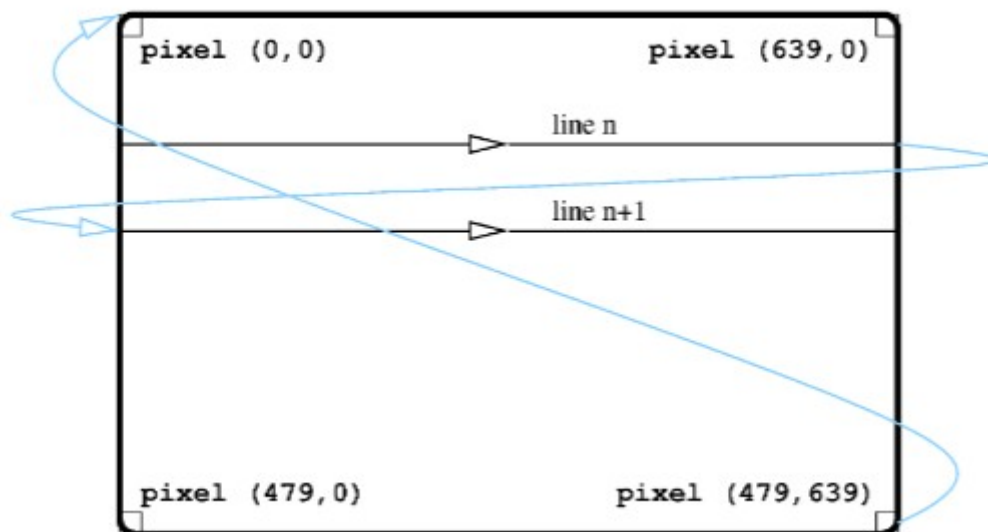
Στην εικόνα φαίνεται η διεπαφή που παρέχει η πλακέτα Artix-7 για την επικοινωνία με την χρήση θύρας VGA. Η θύρα αποτελείται συνολικά από 15 ακίδες, όπου οι 6 από αυτές, τρεις για τα αντίστοιχα (1, 2, 3), και άλλες τρεις για τις σχετικές γειώσεις (6, 7, 8), αντιστοιχούν στα τρία βασικά χρώματα, Κόκκινο, Πράσινο και Μπλε. Παρόλο που με την διεπαφή μας δίνεται η επιλογή να εμφανίσουμε 4096 χρώματα, στα πλαίσια της εργασίας, και για λόγους οικονομίας στη μνήμη, η υλοποίηση μας του ελεγκτή θα υποστηρίξει την κωδικοποίηση οκτώ χρωμάτων. Συγκεκριμένα, θα υλοποιήσουμε την VRAM χρησιμοποιώντας τρεις Block RAM (BRAM), μία για κάθε βασικό χρώμα. Αν η BRAM έχει 1 σε ένα σημείο της δηλώνει την ύπαρξη του χρώματός της στο συγκεκριμένο pixel, αλλιώς δηλώνει την απουσία του. Λόγω του τρόπου που θέλουμε να υλοποιήσουμε το κύκλωμα, η έξοδος του κάθε χρώματος θα αντιστοιχείται σε όλα τα διαθέσιμα pins του χρώματος. Λόγω περιορισμού χώρου σε συνεχόμενο Block της RAM, θα αποθηκεύσουμε μια συμπιεσμένη εκδοχή της εικόνας που θα προβάλλουμε (128x96 αντί 640x480), την οποία θα μεγενθύνουμε τεχνητά





Horizontal and Vertical VGA Synchronisation

Τα σήματα HSYNC και VSYNC είναι υπεύθυνα για τον χρονισμό της οριζόντιας και κατακόρυφης συντεταγμένης αντίστοιχα. Οι κυματομορφές τους είναι πανομοιότυπες, όπως φαίνεται και στο σχήμα παραπάνω. Η οθόνη θα είναι ενεργή για χρόνο D οριζοντίως και R κατακόρυφως, ενώ θα είναι απενεργοποιημένη για χρόνο C μετά και E πριν το HSYNC, καθώς και Q μετά και S πριν το VSYNC. Ο χρονισμός των παραπάνω διαστημάτων εξαρτάται από την ανάλυση και τον ρυθμό ανανέωσης της οθόνης. Η υλοποίηση μας υποθέτει ανάλυση 640 x 480 και ρυθμό ανανέωσης 60Hz. Η εναλλαγή των εξόδων θα επιτευχθεί με τη χρήση FSM σε δύο διαφορετικές μονάδες, μια για το HSYNC και μια για το VSYNC.



Pixel Writing Priority

Στο παραπάνω σχήμα φαίνεται η σειρά με την οποία θα πρέπει να γράφονται τα pixel στην οθόνη. Οι μονάδες VSYNC και HSYNC που αναφέρθηκαν πριν θα περιέχουν και έναν μετρητή οριζόντιων pixel HPIXEL και έναν κατακόρυφο μετρητή pixel VPIXEL αντίστοιχα. Ο μετρητής HPIXEL θα αυξάνεται συνεχώς κατά την εγγραφή δεδομένων και θα μηδενίζεται μετά την εγγραφή μιας γραμμής, ενώ ο μετρητής VPIXEL θα αυξάνεται κατά την εγγραφή μιας γραμμής και θα μηδενίζεται μετά την εγγραφή όλης της εικόνας. Η υλοποίηση λαμβάνει υπόψη ότι η εικόνα που είναι αποθηκευμένη είναι μικρότερη από αυτή που πρέπει να προβληθεί, για αυτό θα καθυστερήσει τεχνητά την ταχύτητα αύξησης των μετρητών, καθιστώντας την πέντε φορές μικρότερη.

Δίπλα φαίνεται η εικόνα που θα πρέπει να προβληθεί στην οθόνη. Θα αλλάξουμε ελαφρώς τα δεδομένα στις αρχικές της διευθύνσεις που κωδικοποιούν ένα εύρος χρωμάτων για να επιβεβαιώσουμε τον σωστό έλεγχο της ram. Επιλέχθηκε ασπρόμαυρη εικόνα γιατί είναι ευκολότερο να τη μεταφέρουμε στη ram μέσω script. Θα σταλθεί και ένα δεύτερο αρχείο nram που περιέχει μία εναλλακτική εικόνα.



- **Μέρος 1ο: Υλοποίηση VRAM**

Για να υλοποιήσουμε τη μνήμη VRAM που θα περιέχει την εικόνα που θα απεικονίζουμε στην εικόνα, μπορούμε να χρησιμοποιήσουμε την Block RAM (BRAM) που περιλαμβάνει η FPGA. Συγκεκριμένα, θα χρησιμοποιήσουμε τρία τμήματα της BRAM, με 18 Kbyte χώρου το καθένα, τα οποία θα περιέχουν πληροφορίες για το αν το pixel που αντιστοιχεί στη διεύθυνση που θα κοιτάζουμε έχει κόκκινο, πράσινο ή μπλέ χρώμα αντίστοιχα. Δηλαδή ανάλογα των περιεχομένων της μνήμης, τα χρώματα θα αντιστιχθούν ως εξής στην έξοδο:

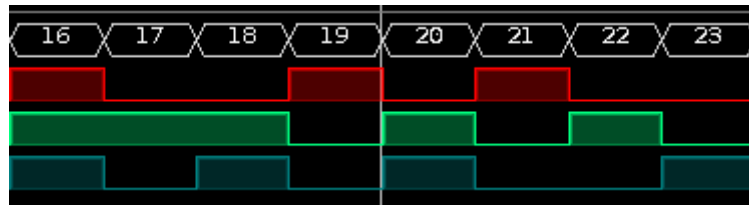
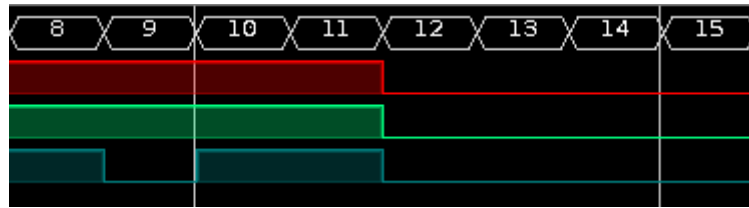
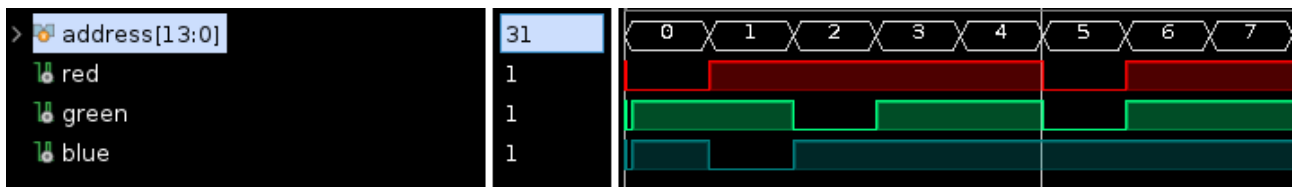
BRAM (Red)	BRAM (Green)	BRAM (Blue)	Displayed Color
0	0	0	Μαύρο
0	0	1	Μπλέ
0	1	0	Πράσινο
0	1	1	Κυανό
1	0	0	Κόκκινο
1	0	1	Μόβ
1	1	0	Κίτρινο
1	1	1	Άσπρο

PartA Possible Color Combination Table

Παραπάνω αναφέρθηκε ότι θα χρησιμοποιηθούν τρία τμήματα της BRAM, με 18 Kbyte χώρου το καθένα. Προφανώς όμως δεν αρκούν για να αποθηκεύσουμε $640 \times 480 \times 3 \text{ bits} (= 307200 \text{ bits} \times 3 = 115.2 \text{ KBytes} \times 3 \text{ επειδή θα χρησιμοποιήσουμε 3 χρώματα})$ στη μνήμη. Λόγω απλότητας της υλοποίησης και λόγω έλλειψης εύκολα προσπελάσιμης μνήμης θα αποθηκεύσουμε μόνο $128 \times 96 \text{ bits} (= 640 \times 480 / 5 \text{ bits})$ ανα τμήμα και θα μεγενθύνουμε τεχνητά την εικόνα κρατώντας σταθερή τη διεύθυνση που θα μεταφέρουμε στη VRAM (και οριζόντια και κατακόρυφα), μέχρι να γίνει εγγραφή 5 pixel (όταν γίνει εγγραφή 5 pixel οριζόντια θα αλλάξει το “οριζόντιο” κομμάτι της διεύθυνσης, όμοια κατακόρυφα). Λεπτομέρειες της υλοποίησης φαίνονται σε μονάδες που περιγράφονται παρακάτω.

Για την προσπέλασή της χρησιμοποιούμε τρεις μονάδες RAMB18E1, τις οποίες θα ρυθμίσουμε για λόγους απλότητας να επιστρέφουν μία τιμή για κάθε διεύθυνση που τις δίνουμε (θέτοντας τη παράμετρο READ_WIDTH_A ίση με 1). Επειδή η μονάδα επιστρέφει 16 bit στη θύρα DOADO, θα αναθέσουμε στην έξοδο της VRAM μόνο τα τελευταία bit από τις εξόδους των μονάδων. Από τη στιγμή που δεν θα χρειαστεί να γράψουμε κάτι στη μνήμη, θα αφήσουμε ανοιχτή και στις τρεις μονάδες μόνο τη θύρα A, και μόνο για ανάγνωση. Παρόλο που δεν θα χρειαστεί να χρησιμοποιήσουμε θύρες όπως η DOPADOP (που είναι για parity, εδώ δεν χρειαζόμαστε έλεγχο) και DOBDO (που είναι για την 2η θύρα), θα περάσουμε ενδεικτικές εισόδους για να αποφύγουμε προειδοποιήσεις του vivado.

Για την επαλήθευση αρκεί να ελέγξουμε αν για κάποιες συγκεκριμένες διευθύνσεις, οι τιμές που υιοθετούν οι μεταβλητές εξόδου είναι οι αναμενόμενες. Αρκεί να προσωμοιώσουμε το κύκλωμα σε επίπεδο συμπεριφοράς (πρασδιορίζεται στην εκφώνηση).



PartA Behavioural Simulation for Different Adresses

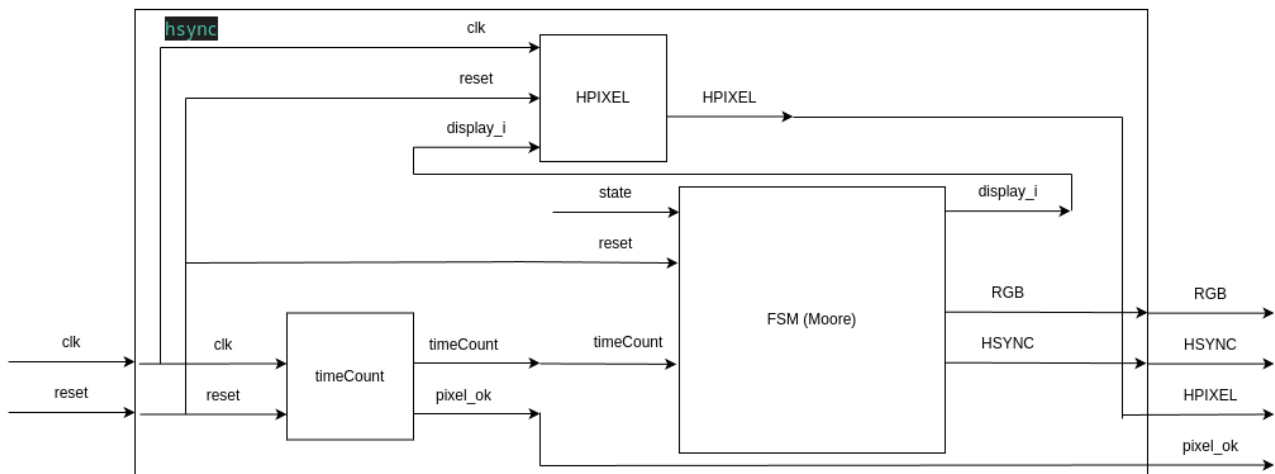
Οι παραπάνω κυματομορφές εμφανίζονται για τις εξείς αρχικές διευθύνσεις:

- Η μονάδα που αποθηκεύει πληροφορίες για το κόκκινο χρώμα έχει **DC290FDE** στις αρχικές της διευθύνσεις.
- Η μονάδα που αποθηκεύει πληροφορίες για το πράσινο χρώμα έχει **E4570FDB** στις αρχικές της διευθύνσεις.
- Η μονάδα που αποθηκεύει πληροφορίες για το μπλέ χρώμα έχει **BA950DFD** στις αρχικές της διευθύνσεις.

- **Μέρος 2ο: Υλοποίηση HSYNC και Οριζόντιου Μετρητή Pixel**

Όπως αναφέρθηκε και στην εισαγωγή, τα σήματα HSYNC και VSYNC είναι υπεύθυνα για τον κατακόρυφο και κάθετο χρονισμό αντίστοιχα. Θα χρειαστούμε δηλαδή δύο μονάδες που θα ελέγχουν το σωστό χρονισμό των παραπάνω σημάτων.

Παρακάτω φαίνεται ένα διάγραμμα ροής της μονάδας. Οι έξοδοι περιλαμβάνουν το σήμα συγχρονισμού της οθόνης HSYNC, ένα σήμα RGB που σηματοδοτεί πότε η οθόνη είναι ενεργή, έναν μετρητή HPIXEL που σηματοδοτεί σε ποιο σημείο της VRAM πρέπει να κοιτάξουμε για να προβάλλουμε σωστά το επόμενο pixel, και ένα σήμα pixel_ok που θα επιτρέψει στη μονάδα VSYNC να έχει μία παρόμοια λειτουργία VPIXEL.

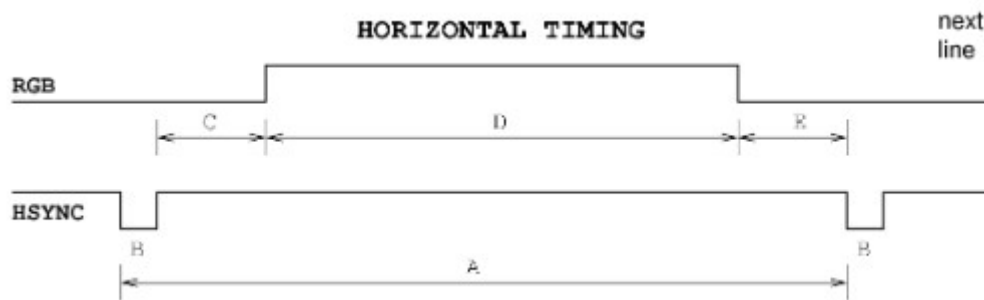


PastB Dataflow Diagramm

Στον παρακάτω πίνακα φαίνονται οι τιμές των διαστημάτων, καθώς και οι κύριοι έξοδοι της μονάδας HSYNC για κάθε διάστημα. Ακολουθείται απο ένα σχήμα το οποίο δείχνει γραφικά αυτά τα διαστήματα. Η έξοδος RGB σηματοδοτεί πότε θα πρέπει να είναι ανοιχτή η οθόνη (θα είναι κλειστή όταν παίρνει τη τιμή 0). Η έξοδος HSYNC προφανώς χρειάζεται για τον σωστό συγχρονισμό με την οθόνη, και έχει ενεργό 0. Προφανώς οι τιμές που παρουσιάζονται προϋποθέτουν μια οθόνη VGA με ανάλυση 640 x 480 και ρυθμό ανανέωσης 60Hz.

Διάστημα	Περιγραφή	HSYNC	RGB	Χρόνος
A	Scanline Time	-	-	32μsec
B	HSYNC Pulse Width	0	0	3.84μsec
C	Back Porch	1	0	1.92μsec
D	Display Time	1	1	25.6μsec
E	Front Porch	1	0	0.640μsec

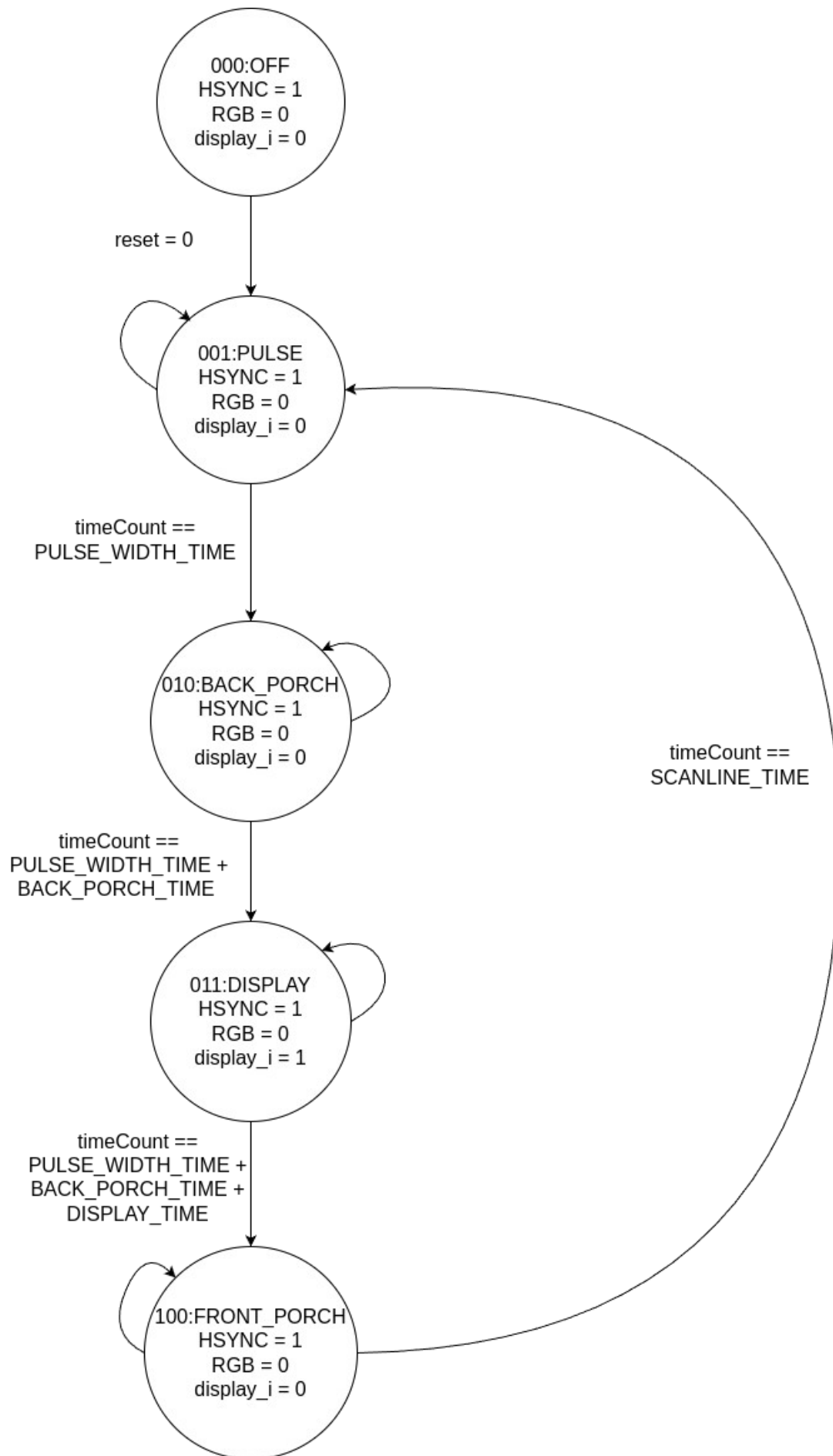
Part B Timings for Each State



Part B Horizontal VGA Synchronisation

Όπως φαίνεται και παραπάνω, το σύστημα μπορεί να εκφραστεί με τη χρήση τεσσάρων καταστάσεων, οι οποίες εναλλάσσονται ανάλογα με τον χρόνο. Μπορούμε δηλαδή να χρησιμοποιήσουμε μια μηχανή πεπερασμένων καταστάσεων (FSM) για τη σωστή οδήγηση των εξόδων. Παρακάτω φαίνονται πληροφορίες σχετικά με το κάθε state, καθώς και ένα διάγραμμα που περιέχει τη δομή της FSM:

- **OFF(000):** Το state στο οποίο η μηχανή επαναφέρεται μετά το πάτημα του κουμπιού reset. Υπάρχει με σκοπό να αποφευχθεί η επιμύκηση του σήματος HSYNC μετά το πάτημα του reset, που θα γινόταν αν επαναφερόμασταν στο PULSE. Κατα τη κανονική λειτουργία του δεν θα επαναφέρεται σε αυτό το state, και θα προχωράει όταν το reset ξαναμηδενιστεί
- **PULSE(001):** Σε αυτό το state θα δημιουργήσουμε τον παλμό HSYNC για να χρονίσουμε την οριζόντια συντεταγμένη στην οθόνη. Κατα τη διάρκεια του παλμού η οθόνη θα είναι κλειστή. Θα μεταβούμε στο BACK PORCH state όταν ο μετρητής timeCount γίνει ίσος με PULSE WIDTH TIME
- **BACK_PORCH(010):** Η οθόνη θα είναι απενεργοποιημένη για ένα χρονικό διάστημα μετά τον παλμό. Θα μεταβούμε στο DISPLAY state όταν ο μετρητής timeCount γίνει ίσος με PULSE WIDTH TIME + BACH PORCH TIME.
- **DISPLAY(011):** Στο state αυτό η οθόνη θα είναι ενεργοποιημένη και θα γράφουμε πάνω της. Θα ενεργοποιηθεί και μια έξοδος display_i για να σηματοδοτήσει στον μετρητή HPIXEL ότι η εγγραφή είναι ενεργή (λεπτομέρειες για την υλοποίησή του παρακάτω). Θα μεταβούμε στο FRONT PORCH state όταν ο μετρητής timeCount γίνει ίσος με PULSE WIDTH TIME + BACH PORCH TIME + DISPLAY TIME.
- **FRONT_PORCH(100):** Όμοια με το BACK PORCH, η οθόνη θα παραμείνει κλειστή πριν τον επόμενο παλμό HSYNC. Θα ξαναγυρίσουμε στο PULSE όταν ο μετρητής timeCount γίνει ίσος με SCANLINE TIME (ισοδύναμο με PULSE WIDTH TIME + BACH PORCH TIME + DISPLAY TIME + FRONT PORCH TIME).



PartB FSM Diagramm

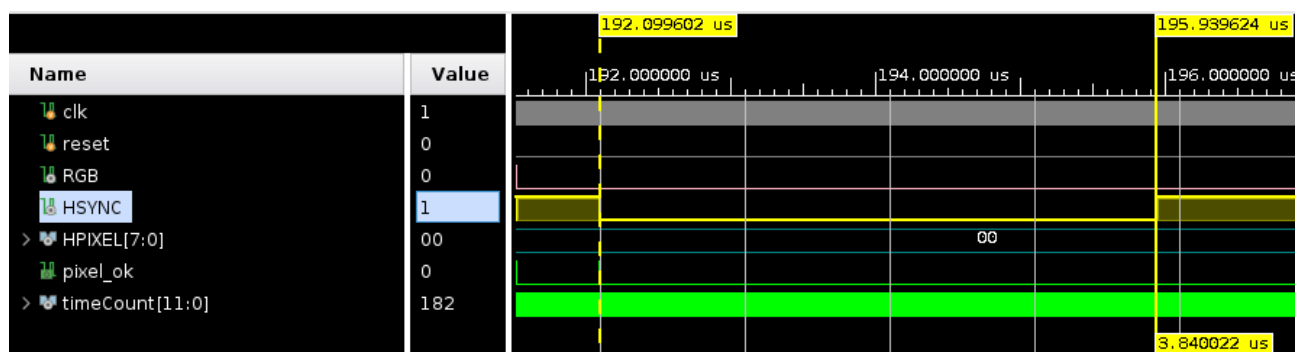
Η FSM είναι τύπου moore και υλοποιήθηκε με τη χρήση 2 always block, ενός συνδιαστικού που θα είναι υπεύθυνο για τις εξόδους της (προφανώς αυτό περιλαμβάνει και την ανάθεση του επόμενου state), και ενός ακολουθιακού, που αναθέτει συνεχώς στο state το επόμενο state (που αναφέρθηκε προηγουμένως). Οι παραπάνω επιλογές αρκούν για την οδήγηση των εξόδων σε σωστούς χρόνους, άρα και τη σωστή απεικόνιση της εικόνας στην οθόνη.

Για την εναλλαγή των state θα υλοποιήσουμε έναν απλό μετρητή κύκλων ρολογιού. Αυτός θα αποτελεί είσοδο στην FSM, και θα σηματοδοτεί την αλλαγή state όταν γίνει ίσος με προκαθορισμένες τιμές που έχουν οριστεί με τη χρήση parameter. Όταν ο μετρητής μεγιστοποιηθεί (όταν δηλαδή περάσει το κομμάτι του χρόνου που καθορίζεται απο το scanline time), ο μετρητής γίνεται ίσος με ένα.

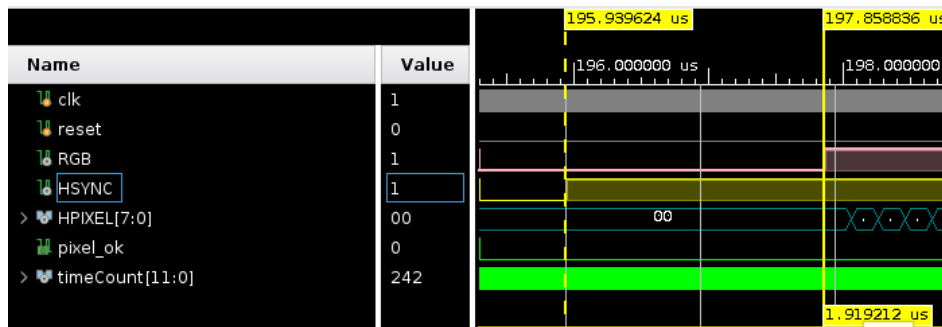
Το σήμα pixel_ok εξαρτάται επίσης απο τον μετρητή και σηματοδοτεί στο VSYNC μια πλήρη ανάγνωση γραμμής. Δηλαδή όταν ο μετρητής γίνει ίσος με το scanline time, τότε το σήμα θα γίνει ίσο με ένα για έναν κύκλο.

Θα χρειαστούμε ένα ακόμα ακολουθιακό always block για να υλοποιήσουμε τον μετρητή HPIXEL. Για να “αποσυμπιέσουμε” την εικόνα (πρέπει να γίνει display με ανάλυση 640 x 480, ενώ στην μνήμη είναι αποθηκευμένη με ανάλυση 128 x 96), θα μετρήσουμε την εγγραφή 5 pixel στην οθόνη πριν αλλάξουμε την τιμή του HPIXEL κατα 1, δηλαδή θα αυξάνεται κάθε 20 κύκλους ρολογιού. Αυτή η αύξηση θα γίνεται μόνο όταν βρισκόμαστε στο display state (όταν το σήμα display_i είναι ίσο με 1), απο το οποίο όταν φεύγουμε, ο μετρητής θα μηδενίζεται.

Για να επαληθεύσουμε τη σωστή λειτουργία της μονάδας, θα πρέπει να ελέγξουμε οτι οι χρόνοι μεταξύ κάθε κατάστασης της FSM είναι σωστοί, ή τουλάχιστον δεν έχουν απόκλιση απο τους επιθυμητούς χρόνους παραπάνω απο 2nsec. Παρακάτω φαίνονται screenshots απο post-implementation timing simulation στα οποία μετριοούνται οι χρόνοι για κάθε state, καθώς και ένας πίνακας που περιέχει τους πραγματικούς χρόνους σε σύγκριση με τους αναμενόμενους. Θα παρατηρήσουμε πως τα σφάλματα μεταξύ του αναμενόμενου και του πραγματικού χρόνου μεταξύ κάθε state βρίσκονται εντός του ορίου ανοχής των 2nsec.



PartB Post-Implementation Timing Simulation, Pulse State Timings



PartB Post-Implementation Timing Simulation, Back Porch State Timings



PartB Post-Implementation Timing Simulation, Display State Timings

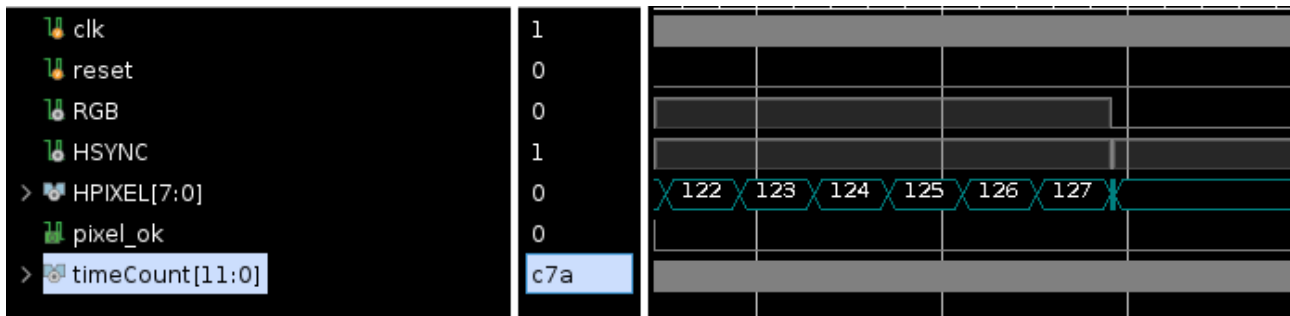


PartB Post-Implementation Timing Simulation, Front Porch State Timings

Περιγραφή	Αναμενόμενος Χρόνος	Πραγματικός Χρόνος	Διαφορά
HSYNC Pulse Width	3.84μsec	3.840022μsec	0.022nsec
Back Porch	1.92μsec	1.919212μsec	0.788nsec
Display Time	25.6μsec	25.6μsec	0nsec
Front Porch	0.64μsec	0.640766μsec	0.766nsec

PartB Post-Implementation Timing Simulation, Differences Between Expected and Real Times

Για την δοκιμή του μετρητή HPIXEL, αρκεί να επιβεβαιώσουμε ότι όντως αυξάνεται μόνο όσο είμαστε στο display state και μηδενίζεται εκτός του, κάτι που φαίνεται και από τα παραπάνω στιγμιότυπα, και ότι μεγιστοποιείται με τιμή 127 (128 pixel αποθηκευμένα οριζόντια, και ο μετρητής αρχίζει από το 0). Αυτό φαίνεται στο στιγμιότυπο παρακάτω:

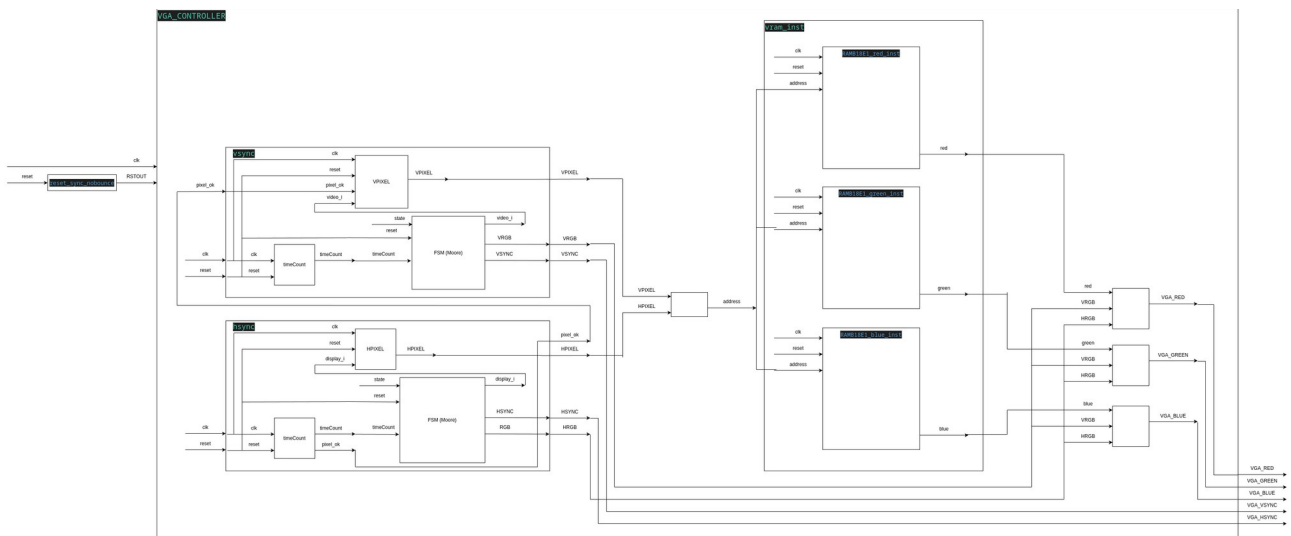


PartB Post-Implementation Timing Simulation, HPIXEL Max Value

Αξίζει να σημειωθεί ότι παραπάνω φαίνεται και η σωστή λειτουργία του pixel_ok, καθώς ανεβαίνει για έναν κύκλο μετά το BACK PORCH, όπως περιμέναμε. Το testbench δεν περιγράφεται αναλυτικά, καθώς απλώς οδηγεί συνεχώς το ρολόι και στην αρχή το reset στέλνοντας έναν μικρό παλμό. Προειδοποιήσεις του νίναδο αφορούσαν παράλληλη σύνθεση και virtual grid που μπορούμε να αγνοήσουμε, και προειδοποιήσεις σχετικά με τα constraints που δεν μας αφορούν σε αυτό το στάδιο.

- **Μέρος 3ο: Υλοποίηση VSYNC και Κατακόρυφου Μετρητή Pixel - Ολοκλήρωση του Ελεγκτή/Οδηγού VGA.**

Σε αυτό το κομμάτι θα υλοποιήσουμε τη μονάδα VSYNC, που θα ελέγχει τα αντίστοιχα σήματα χρονισμού και έναν μετρητή VPIXEL, που σηματοδοτεί πόσα pixel έχουμε γράψει κάθετα, η πόσες γραμμές απο pixel έχουμε γράψει στην οθόνη (λαμβάνοντας φυσικά υπόψη πως τα δεδομένα αποθηκευμένα στη μνήμη είναι συμπιεσμένα), καθώς και μια μονάδα που θα συνδέει τα σήματα HSYNC και VSYNC με την έξοδο που δρομολογείται στην οθόνη, και που θα χρησιμοποιεί τα HPIXEL VPIXEL και τα RGB σήματα για να εξάγει και να δρομολογήσει τα σωστά δεδομένα απο τη μνήμη προς την έξοδο. Παρακάτω φαίνεται ένα ολικό διάγραμμα ροής του κυκλώματος όπως περιγράφηκε:



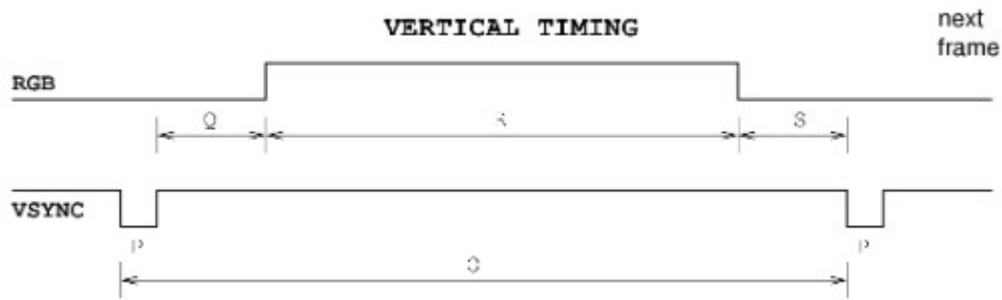
Part C Flow Diagramm

Η μονάδα VSYNC είναι όμοια με την μονάδα HSYNC που περιγράφηκε παραπάνω, οπότε εδώ θα δωθεί έμφαση σε σημεία στα οποία οι δύο μονάδες διαφέρουν

Στον παρακάτω πίνακα φαίνονται οι τιμές των διαστημάτων, καθώς και οι κύριοι εξοδοι της μονάδας VSYNC για κάθε διάστημα (οι οποίες λειτουργούν όμοια με αυτές της μονάδας HSYNC). Ακολουθείται απο ένα σχήμα το οποίο δείχνει γραφικά αυτά τα διαστήματα.

Διάστημα	Περιγραφή	HSYNC	RGB	Χρόνος
O	Total Frame Time	-	-	16.672msec
P	VSYNC Pulse Width	0	0	64μsec
Q	Back Porch	1	0	928μsec
R	Active Video Time	1	1	15.36msec
S	Front Porch	1	0	320μsec

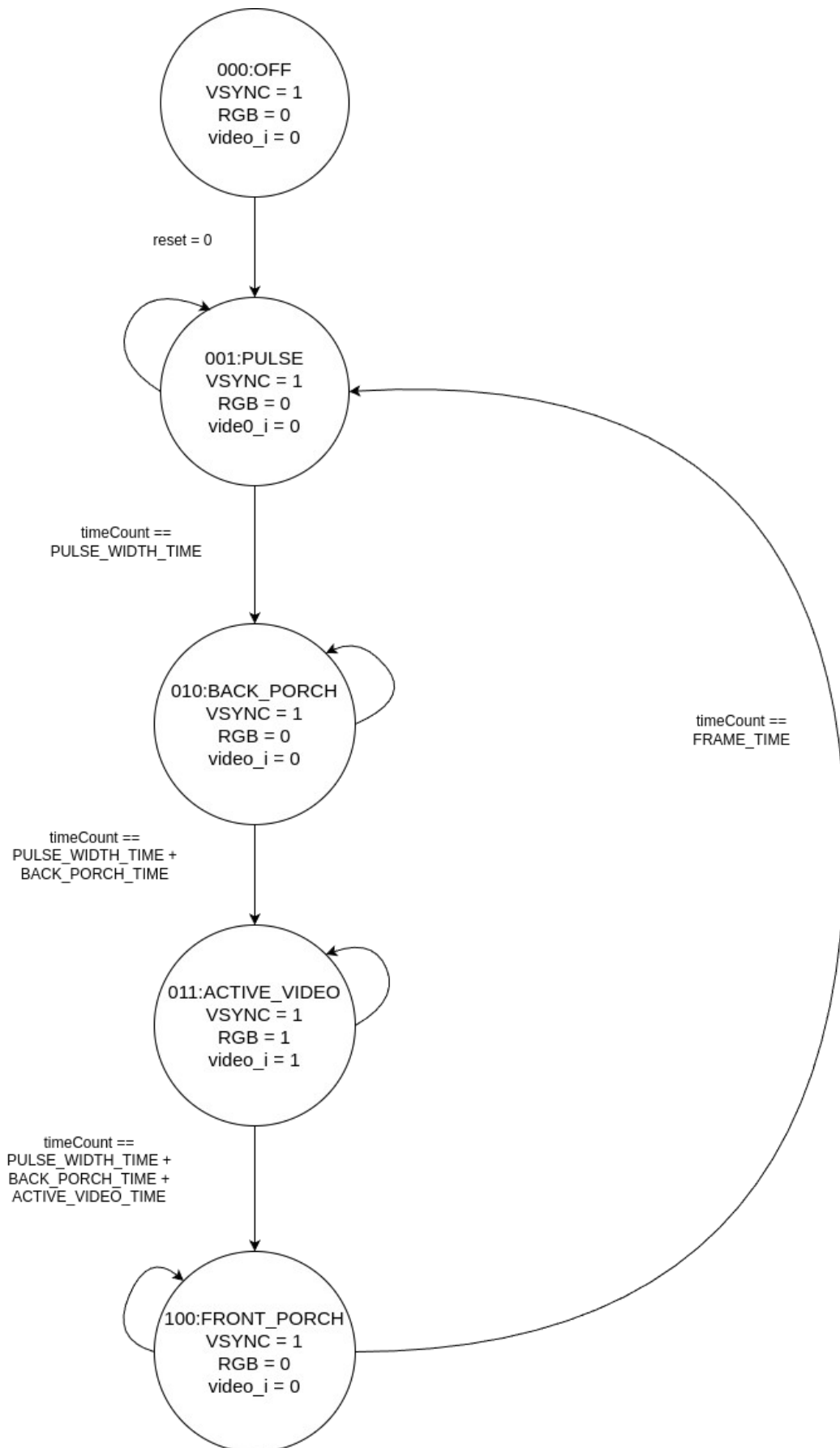
Part C Timings for Each State



Part C Horizontal VGA Synchronisation

Το σωστό χρονισμό θα υλοποιήσουμε ξανά με χρήση FSM με δύο always blocks όπως και παραπάνω. Παρακάτω φαίνονται διαφορές μεταξύ των state αυτής της FSM και αυτής που υλοποιήθηκε παραπάνω, και ένα διάγραμμα που περιέχει τη δομή της FSM. Ο αντίστοιχος μετρητής timeCount λειτουργεί επίσης όμοια με πάνω, όπως και η λογική πίσω από την αλλαγή από κάθε στο επόμενο state (προφανώς τα ονόματα και οι χρόνοι ορισμένοι με parameter είναι διαφορετικοί, αλλά έχουν οριστεί με όμοια λογική με πάνω).

- **OFF(000):** Λειτουργεί όμοια με παραπάνω.
- **PULSE(001):** Θα δημιουργήσουμε τον παλμό VSYNC για να χρονίσουμε την κατακόρυφη συντεταγμένη στην οθόνη. Κατά τη διάρκεια του παλμού η οθόνη θα είναι κλειστή.
- **BACK_PORCH(010):** Η οθόνη θα είναι απενεργοποιημένη για ένα χρονικό διάστημα μετά τον παλμό, όπως και παραπάνω.
- **ACTIVE_VIDEO(011):** Στο state αυτό η οθόνη θα είναι ενεργοποιημένη και θα γράφουμε πάνω της. Θα ενεργοποιηθεί και μια έξοδος video_i για να σηματοδοτήσει στον μετρητή VPIXEL ότι η εγγραφή είναι ενεργή (λεπτομέρειες για την υλοποίησή του παρακάτω).
- **FRONT_PORCH(100):** Όμοια με το BACK PORCH, η οθόνη θα παραμείνει κλειστή πριν τον επόμενο παλμό VSYNC.



PartC FSM Diagramm

Για την υλοποίηση του μετρητή VPIXEL, θα χρησιμοποιήσουμε πάλι ένα ακολουθιακό always block. Για να λειτουργήσει σωστά θα μετρήσουμε την εγγραφή 5 γραμμών πριν αλλάξουμε τη τιμή του, αποσυμπιέζοντας την εινόνα που είναι αποθηκευμένη στη μνήμη. Αυτή η αύξηση θα γίνεται μόνο όταν βρισκόμαστε στο active video state (όταν το σήμα video_i είναι ίσο με 1). Εντοπίζουμε την αλλαγή γραμμής με τη χρήση της εισόδου pixel_ok, που παρέχεται απο την μονάδα HSYNC. Εκτός του video state ο μετρητής μηδενίζεται.

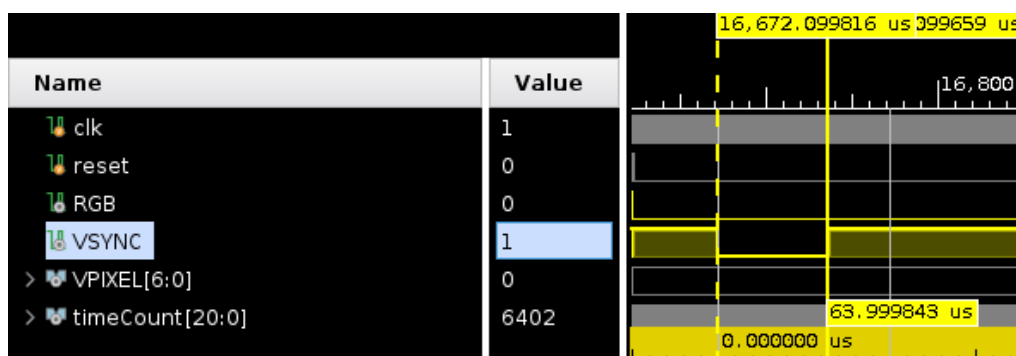
Για την ολοκλήρωση του VGA_DRIVER μας μένει η σωστή οδήγηση των εξόδων και τον εισόδων στις μονάδες. Στα constraints, εκτός απο το clk και το reset, που θα οδηγηθούν όπως σε προηγούμενα εργαστήρια, θα προσθπесουμε τα εξής:

- Οι εξόδοι VGA_HSYNC και VGA_VSYNC θα οδηγηθούν στα pins B11 και B12 αντίστοιχα.
- Για το κάθε χρώμα υπάρχουν τέσσερα pins στις οποίες μπορούμε να τα οδηγήσουμε, το καθένα με διαφορετική αντίσταση, το οποίο επιτρέπει στη προβολή χρωμάτων διαφορετικής έντασης, καθώς και σε μεγάλο εύρος αποχρώσεων. Εμάς μας αρκεί η προβολή βασικών χρωμάτων, οπότε θα οδηγήσουμε τις εξόδους των χρωμάτων και στα τέσσερα αντίστοιχα pins. Για παράδειγμα, θα οδηγήσουμε την έξοδο VGA_RED στα pins A3, B4, C5 και A4 (όμοια τα υπόλοιπα).

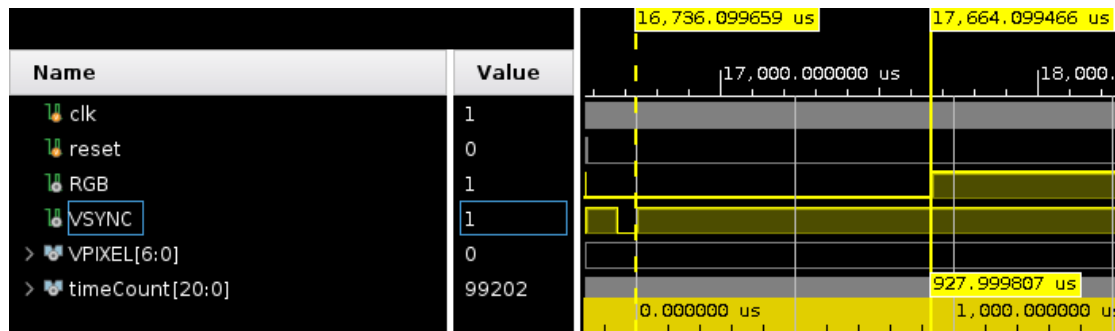
Για τη σωστή εξαγωγή δεδομένων απο τη μνήμη, θα κάνουμε concatenate το HPIXEL και το VPIXEL με την έκφραση assign address = {VPIXEL, HPIXEL};. Επίσης οι εξόδοι VGA_RED, VGA_GREEN και VGA_BLUE δεν πρέπει να περιέχουν δεδομένα όταν η οθόνη είναι κλειστή, δηλαδή όταν είτε το HRGB είτε το VRGB είναι ίσα με 0, οι τιμές τους θα είναι επίσης ίσες με 0. Για το reset θα χρησιμοποιήσουμε έναν συγχρονιστή και ένα antibounce, που έχουμε υλοποιήσει σε προηγούμενο εργαστήριο.

Για να επαληθεύσουμε τη σωστή λειτουργία της μονάδας, αρχικά θα πρέπει να ελέγξουμε, όπως και για το HSYNC, οτι οι χρόνοι στο VSYNC μεταξύ κάθε κατάστασης της FSM είναι σωστοί, ή τουλάχιστον δεν έχουν απόκλιση απο τους επιθυμητούς χρόνους παραπάνω απο 2nsec. Παρακάτω φαίνονται screenshots απο post-implementation timing simulation στα οποία μετριοούνται οι χρόνοι για κάθε state, καθώς και ένας πίνακας που περιέχει τους πραγματικούς χρόνους σε σύγκριση με τους αναμενόμενους. Θα παρατηρήσουμε πως τα σφάλματα μεταξύ του αναμενόμενου και του πραγματικού χρόνου μεταξύ κάθε state βρίσκονται εντός του ορίου ανοχής των 2nsec.

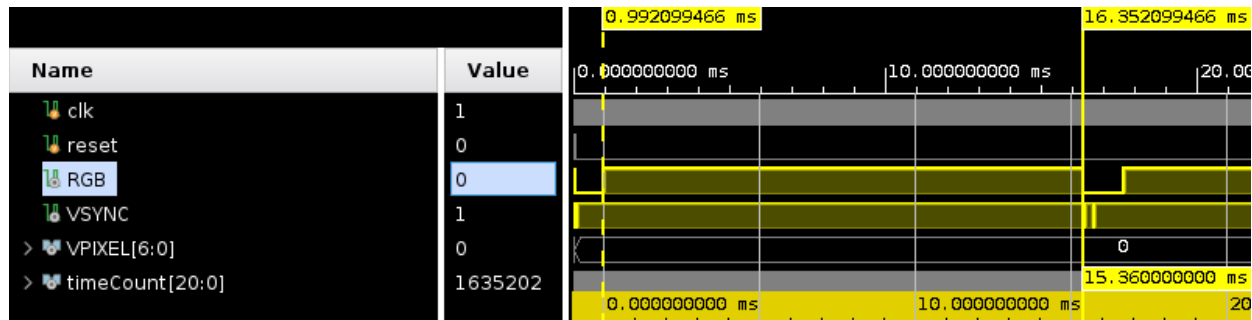
Η δοκιμή αρχικά θα γίνει με την μονάδα VSYNC μεμονομένα, με όμοιο testbench με αυτό που χρησιμοποιήσαμε στο HSYNC, για να έχουμε πιο εύκολα αναγνώσιμες εξόδους. Θα οδηγήσουμε την είσοδο pixel_ok στο 0, καθώς είναι βολικότερο να ελέγξουμε την ορθή της λειτουργία του μετρητή VPIXEL όταν θα ελέγχουμε ολικά το κύκλωμα, γιατί το σήμα pixel_ok θα οδηγείται αυτόματα απο το HSYNC.



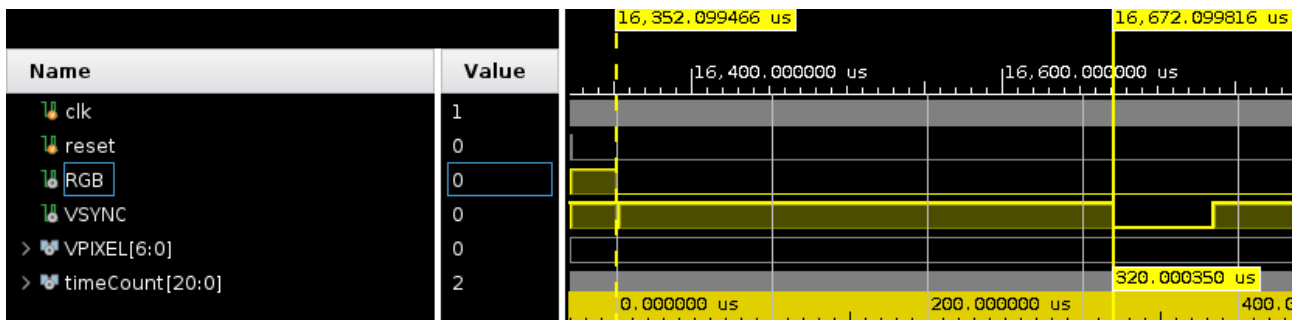
PartC Post-Implementation Timing Simulation, Pulse State Timings



PartC Post-Implementation Timing Simulation, Back Porch State Timings



PartC Post-Implementation Timing Simulation, Active Video State Timings

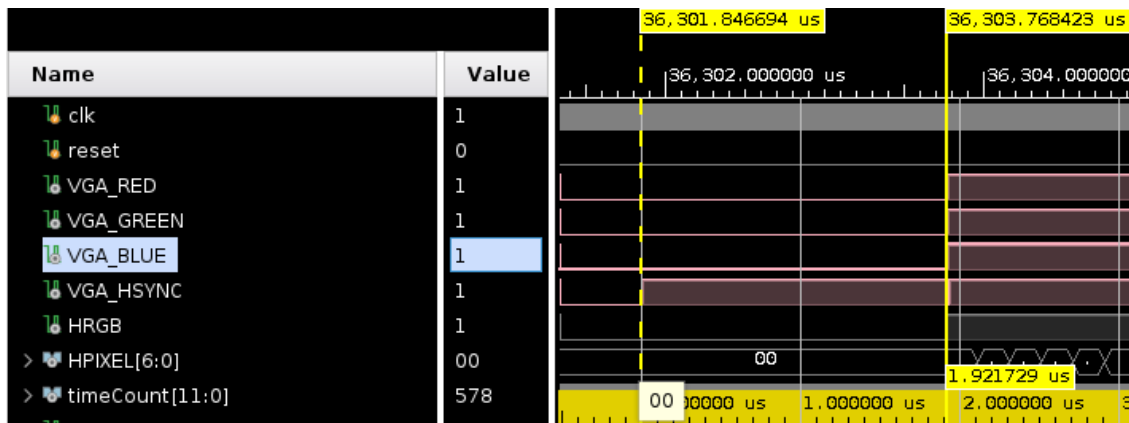


PartC Post-Implementation Timing Simulation, Front Porch State Timings

Περιγραφή	Αναμενόμενος Χρόνος	Πραγματικός Χρόνος	Διαφορά
HSYNC Pulse Width	64μsec	63.999843μsec	0.157nsec
Back Porch	928μsec	927.999807μsec	0.193nsec
Display Time	15.36msec	15.36msec	0nsec
Front Porch	320μsec	320.000350μsec	0.350nsec

PartC Post-Implementation Timing Simulation, Differences Between Expected and Real Times

Πρέπει να ελέγξουμε επίσης ότι το top module δεν θα προσθέσει σημαντική καθυστέρηση μεταξύ των σημάτων. Απο τη στιγμή που οι χρονισμοί μεταξύ του ίδιου σήματος εξαρτώνται μόνο απο τη μονάδα HSYNC η VSYNC, και αφού στο top module η λογική μεταξύ HSYNC και VSYNC, όπως και μεταξύ του HRGB και VRGB είναι η ίδια, αρκεί να ελέγξουμε ότι το HSYNC και το HRGB έχουν μικρή διαφορά μεταξύ τους. Αυτό φαίνεται στα παρακάτω στιγμιότυπα.



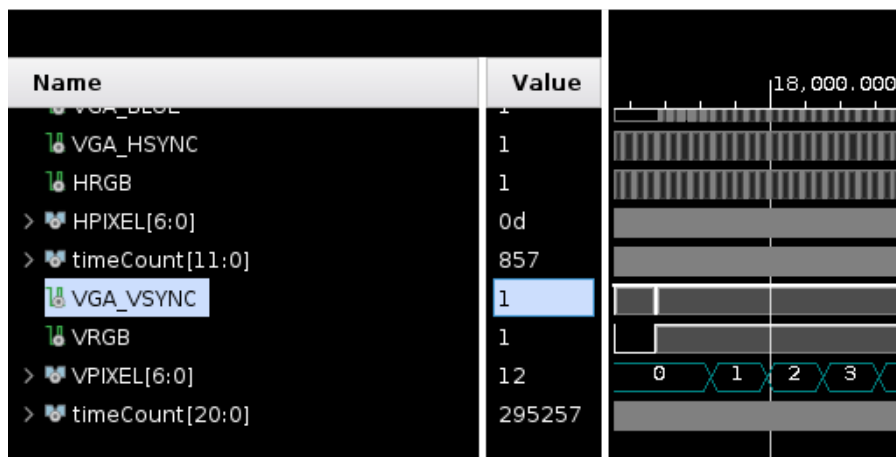
PartC Post-Implementation Timing Simulation (Full Circuit), Back Porch State Timings



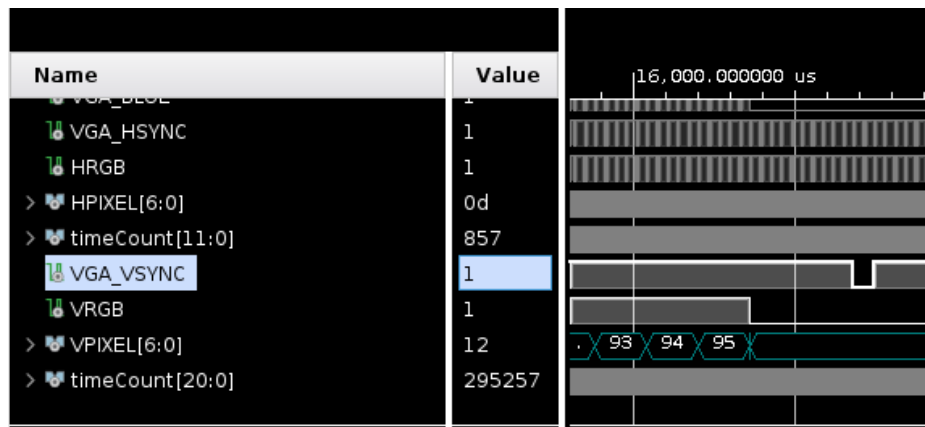
PartC Post-Implementation Timing Simulation (Full Circuit), Front Porch State Timings

Οι παραπάνω έλεγχοι γίνονται μεταξύ του HSYNC και των σημάτων VGA_color, επειδή και τα δύο είναι εξόδοι. Αντίθετα, το σήμα HRGB είναι εσωτερικό της μονάδας HSYNC, οπότε δεν είναι αντιπροσωπευτικό της πραγματικής καθυστέρησης.

Για τον μετρητή VSYNC αρκεί επίσης να ελέγξουμε οτι όντως αυξάνεται μόνο όσο είμαστε στο display state και μηδενίζεται εκτός του, κάτι που φαίνεται και απο τα παραπάνω στιγμιότυπα, και οτι μεγιστοποιείται με τιμή 95.



PartB Post-Implementation Timing Simulation (Full Circuit), VPIXEL Min Value



PartB Post-Implementation Timing Simulation (Full Circuit), VPIXEL Max Value

Κάποια προβλήματα στην υλοποίηση φανερώθηκαν κατά τη διάρκεια της επαλήθευσης. Αρχικά η υλοποίηση αλλαγής των state γινόταν εκτός της fsm, στο ίδιο always block με τον μετρητή. Όταν η τιμή του μετρητή γινόταν ίση με τις κατάλληλες τιμές που όριζαν οι παράμετροι ένας παλμός σε μία μεταβλητή switch στέλνόταν στην FSM, που προκαλούσε την αλλαγή state. Αυτό, εκτός του γεγονότος ότι οδηγεί σε ακριβή κυκλωματική υλοποίηση, οδηγούσε σε καθυστέρηση 128 κύκλων της VSYNC όταν η FSM άλλαζε από PULSE σε BACK PORCH state.

Το επόμενο πρόβλημα αποτελεί και τον λόγο όπου το κύκλωμα δεν λειτουργούσε σωστά στη πλακέτα κατά τη τελική δοκιμή. Το HSYNC και το VSYNC ορίστηκαν ως 8bit μεταβλητές αντί για 7bit, που οδηγούσε κατά δύο περισσότερα bit κατά το concatenation με το address. Ο συνθέτης αφαιρούσε τα δύο “περιττά” πιο σημαντικά bit από το VSYNC, το οποίο φάνέρωσε το λάθος στο σήμα κατά τον έλεγχο.

Αν και δέν φανερώθηκε απευθείας από τις δοκιμές, ανάθεση στο address γινόταν με πολλαπλασιασμό, κάτι που είναι σχετικά αργό. Κατά τη προσπάθεια βελτιστοποίησης του κυκλώματος, η υλοποίηση άλλαξε ώστε να χρησιμοποιεί concatenation.

Αξίζει να σημειωθεί ότι έγιναν εκτενείς δοκιμές στον χρονισμό των σημάτων, και στα δύο στάδια. Μετά την αλλαγή στην FSM που περιγράφηκε παραπάνω, δεν παρατηρήθηκαν λάθη στον χρονισμό

Τέλος, προειδοποιήσεις του vivado αφορούσαν παράλληλη σύνθεση και virtual grid που μπορούμε να αγνοήσουμε.

Πραγματοποιήθηκαν πολλοί προγραμματισμοί στην πλακέτα FPGA, λόγω ενός βασικού λάθους στη κατανόηση της εκφώνησης, και τη λανθασμένη διάγνωση του προβλήματος. Την πρώτη φορά που προγραμματίσα την πλακέτα κατά αυτή την εργασία ήταν κατά τις ώρες πλακέτας, στις οποίες ο χρόνος αρκούσε μόνο για έναν προγραμματισμό. Η εικόνα που φάνηκε στην οθόνη υποδύκνιε λάθος στον χρονισμό, συγκεκριμένα στον χρονισμό του VSYNC. Αυτό οδήγησε στην αλλαγή στην λογική της fsm που αναφέρθηκε παραπάνω.

Οι υπόλοιποι προγραμματισμοί έγιναν κατά τη διάρκεια του τελευταίου εργαστηρίου. Εκεί παρατηρήθηκε πως ο χρόνος εμφάνισης μιας γραμμής της VRAM ήταν ο χρόνος εμφάνισης μιας εικόνας, η καλύτερα πως κάθε γραμμή της εικόνας εκτυπωνόταν σαν ξεχωριστή εικόνα. Το λάθος υπήρχε στη κατανόηση της λειτουργίας των σημάτων HSYNC και VSYNC, και εκ του αποτελέσματος στην υλοποίηση των HPIXEL και VPIXEL (συγκεκριμένα, νόμιζα πως κάθε παλμός του HSYNC αντιπροσώπευε την αλλαγή ενός pixel αντί μιας γραμμής, και του VSYNC τον χρόνο μίας γραμμής αντί του χρόνου της εικόνας, για αυτό άλλαξα το HPIXEL, VPIXEL να SCANLINE TIME και ACTIVE VIDEO TIME αντίστοιχα).

Οι δοκιμές για την αλλαγή του HPIXEL και του VPIXEL έγιναν κυρίως με απευθείας δοκιμή στη πλακέτα λόγω του μικρού χρονικού διαστήματος μέχρι τη λήξη του τελικού

εργαστηρίου. Στη τελευταία δοκιμή κατά τη διάρκεια της εξέτασης, η εικόνα εμφανιζόταν αλλά δεν φαινόταν σταθερή και φαινόταν σαν να ταξιδεύει προς τα πάνω.

Αυτό οφειλόταν σε σχεδιαστικές παραβλέψεις, όπως την άνοδο του μετρητή εκτός του video state, που παρόλο που παρατηρήθηκαν, καθιστούσαν τη λειτουργικότητα στη πλακέτα χειρότερη. Το κύριο λάθος που δεν παρατηρήθηκε κατά την ώρα του εργαστηρίου αναφέρεται αφορά το μήκος των HPIXEL, VPIXEL και αναφέρεται αναλυτικότερα παραπάνω.